

Semantic Layer Workflows for Adaptive Generative AI Editing in the Compute Continuum

Stelios Sotiriadis^{a,*}

^aSchool of Computing and Mathematical Sciences, Birkbeck, University of London, London, United Kingdom

Abstract

Interactive diffusion editing is usually a session, not a single model call. A user generates an image, selects a region, tries alternatives, preserves surrounding content, and may roll back. Full-image regeneration repeats work and often changes non-target regions, while one-off inpainting does not provide reusable semantic state, provenance, or scheduling information.

This paper introduces Semantic Layer Workflow, a workflow model that represents an image as a graph of semantic layers with masks, bounding boxes, labels, edit history, provenance, cache keys, and execution metadata. The workflow chooses among full regeneration, semantic local diffusion, cached re-editing, deterministic projection, and rollback according to edit scope, latency, preservation risk, and available layer state. The aim is not to replace diffusion editing models, but to make iterative editing schedulable and reproducible across heterogeneous compute resources.

The prototype combines Stable Diffusion and SDXL pipelines with GroundingDINO+SAM2 annotation, crop-local diffusion, mask compositing, projection-only color edits, and persisted versions. The evaluation covers 200 paired local executions across 10 checkpoints, four edit categories, and five seeds, plus measured NVIDIA L4 and T4 tiers. On Apple MPS, semantic rendering is 1.788x faster than full regeneration without detector cost, but detector-inclusive first edits average 0.809x because annotation costs 13.543 s. Cached re-edits reach 7.041x speedup. On an NVIDIA L4, detector-inclusive first edits average 4.28x speedup; seven long edit sessions across five model families reduce 303 full-regeneration variants from 2786.7 s to 528.9 s, giving 5.37x mean speedup while preserving non-target pixels under the implemented compositing metrics.

Keywords: Compute continuum, Scientific workflows, Generative AI, Diffusion models, Semantic layers, Provenance, Adaptive execution, Workflow scheduling, Reproducibility, Performance modeling

1. Introduction

Generative AI pipelines are becoming data- and compute-intensive workflows. In interactive image editing, users iterate: generate an image, identify a region, alter an object or color, undo a variant, preserve identity, and compare alternatives. Each iteration may require detector inference, segmentation, denoising, compositing, metric evaluation, and provenance updates. The process therefore has core properties of scientific workflow execution: heterogeneous tasks, data dependencies, intermediate artifacts, repeated execution, and trade-offs between latency, quality, and resource cost [1–3].

Current diffusion editors synthesize well but manage workflow state poorly. Full-image regeneration recomputes everything and often changes unrelated pixels. Manual inpainting gives local control, but depends on user effort and lacks reusable semantic state. Prompt editing, cross-attention control, ControlNet, and instruction-tuned editors improve controllability [4–6], but most pipelines

still lack named regions, edit histories, cached masks, rollback states, and policy-driven execution.

This paper represents iterative diffusion editing as a semantic scientific workflow. The image is an Image DOM: a graph of named semantic layers with provenance, dependency context, and cached execution state. An edit request becomes a workflow decision. Simple color updates may use deterministic projection; texture changes may use semantic local diffusion; global or entangled edits may require full regeneration; repeated edits and rollback can reuse stored layer state.

This framing matters for compute-continuum systems because the workflow exposes information hidden by monolithic generation calls: region size, expected cost, cache availability, dependency risk, provenance constraints, and candidate operators. The evaluation combines a local benchmark, measured GCP L4 and T4 tiers, and resource-sensitivity projections; it is not yet a deployed edge-cloud-HPC scheduler.

We make four contributions.

1. We define a semantic layer workflow model for diffusion editing, including layer graphs, edit operators, prove-

*Corresponding author.

Email address: s.sotiriadis@bbk.ac.uk (Stelios Sotiriadis)

nance records, cache keys, and version states.

2. We formulate adaptive editing as a multi-objective execution policy that trades latency, detector cost, non-target drift, expected quality loss, and user effort.
3. We implement a prototype workflow engine using Stable Diffusion and SDXL checkpoints, GroundingDINO+SAM2 masks, local crop diffusion, deterministic projection, compositing, cached re-edits, and rollback.
4. We evaluate the system with a 200-sample local benchmark across 10 models, four edit categories, and five seeds, plus measured GCP NVIDIA L4 and T4 tiers. The benchmark reports latency, speedup, rollback time, drift, LPIPS, dependency-contact risk, crop/mask characteristics, policy ablations, long-session throughput, model compatibility, and resource-sensitivity projections.

The central empirical result is mixed but useful. Across 200 local samples, semantic rendering excluding detector overhead is 1.788x faster than naive full-image regeneration, but detector-inclusive first edits average only 0.809x. Cached layer edits reach 7.041x average speedup. On the L4 long-session benchmark, repeated semantic execution averages 5.37x detector-inclusive speedup across five model families. The claim is therefore locality, reproducibility, fast re-editing, and versioned execution, not universal first-edit acceleration.

2. Background and Motivation

2.1. Scientific workflows and provenance

Scientific workflow systems coordinate analyses over heterogeneous resources while preserving execution structure, intermediate artifacts, and provenance [1, 7]. They make dependencies explicit, support monitoring and repeated execution, and help reason about performance and reproducibility across accelerators, clouds, edge resources, and distributed storage [3, 8].

Generative AI editing shares these characteristics. A session combines tasks with different resource profiles: open-vocabulary detection and segmentation, accelerator-bound denoising, lightweight compositing, and storage-bound persistence or rollback. It is also stateful: an edit depends on the base image, prompt, seed, checkpoint, scheduler, masks, previous variants, and user intent. Without provenance, results are hard to reproduce or explain.

Provenance is therefore part of the computational model. The workflow records generated artifacts, model and scheduler choices, detector outputs, crop parameters, and reused cache state, aligning with scientific provenance principles [9, 10] and FAIR expectations [11].

2.2. Diffusion models as workflow components

Denoising diffusion probabilistic models generate samples by learning a reverse denoising process from noise to data [12]. Latent diffusion models perform this process in a compressed latent space, making high-resolution text-to-image generation more efficient [13]. Let z_t be a latent at diffusion timestep t , c the conditioning, and $\epsilon_\theta(z_t, t, c)$ the denoising network. A scheduler update can be written abstractly as

$$z_{t-1} = S_t(z_t, \epsilon_\theta(z_t, t, c), \xi_t), \quad (1)$$

where S_t is the numerical scheduler and ξ_t denotes stochastic state when present. Deterministic samplers such as DDIM [14] reduce randomness, but the output remains dependent on scheduler configuration, conditioning, seed, model weights, and intermediate latents.

Editing methods build on diffusion in different ways: SDEdit denoises a noised input under new conditioning [15], Prompt-to-Prompt controls cross-attention [4], DiffEdit estimates semantic masks [16], and ControlNet adds spatial controls [5]. These techniques do not by themselves define a workflow model for persistent semantic regions, cached edits, and rollback.

2.3. Semantic annotation for editable layers

Semantic layer extraction can use cross-attention maps, such as DAAM for Stable Diffusion attention [17], or open-vocabulary detection and segmentation for arbitrary images. GroundingDINO detects text-specified regions [18, 19]; SAM and SAM2 generate masks from boxes or points [20–22]; depth models can add ordering [23].

The prototype uses GroundingDINO+SAM2 because it works without generation-time attention maps. The architecture can also accept DAAM, face landmarks, OCR/layout detectors, and depth estimation as layer sources.

3. Related Work

3.1. Workflow systems for heterogeneous computation

Scientific workflow research studies how to describe, schedule, execute, monitor, and reproduce dependent computations. Pegasus illustrates explicit workflow representation, provenance capture, and heterogeneous task planning [7]; surveys emphasize abstraction, interoperability, monitoring, fault tolerance, data staging, and reproducibility [1, 2]. Diffusion editing has the same systems shape: a workflow graph, expensive tasks, intermediate artifacts, and user-driven branching.

The compute continuum adds placement choices across local accelerators, cloud GPUs, edge devices, and HPC resources. Prior heterogeneous scheduling work, including HEFT, shows that task-resource matching affects makespan [24]. For generative AI, useful features include duration, dependency depth, semantic scope, mask size,

cache availability, and quality risk. This paper exposes those features while leaving full multi-tier scheduling to future work.

3.2. Provenance and reproducible AI pipelines

Provenance is central to scientific workflows because researchers must know how an output was derived and which computation produced each artifact [9]. W3C PROV formalizes entities, activities, and agents [10]. Generative AI needs the same discipline: an image depends on weights, scheduler settings, prompts, seeds, detector outputs, masks, crop geometry, blending, and previous versions.

FAIR reuse requires more than final images [11]. It requires workflow code, model identifiers, configuration, seeds, metrics, and intermediate artifacts. The public artifact includes manuscript source, scripts, reports, metrics, and representative grids; checkpoints are not redistributed because they have separate licenses.

3.3. Diffusion editing and semantic control

Diffusion editing methods improve controllability through image-conditioned denoising, attention control, semantic masks, instruction following, and spatial controls [4–6, 15, 16]. They mostly answer how to produce an edit, not how to manage many edits with cached semantic state.

Segmentation and open-vocabulary grounding provide another route. SAM/SAM2 produce promptable masks [20, 21], and GroundingDINO maps labels to boxes [18]. These methods can construct named layers, but detector output alone is not a workflow; here it is bound to provenance, versioning, policy, and metrics.

3.4. Positioning

The closest ideas are semantic editing, attention-based diffusion control, inpainting, and workflow provenance. This work differs by making the semantic layer graph a persistent execution object: a cacheable artifact for scheduling, repeated editing, rollback, and reproducibility. It does not propose a new denoising or segmentation model; it proposes a workflow model for using existing components.

4. Problem Formulation

4.1. Editing as a semantic workflow

Let an editing session be represented as a directed acyclic workflow

$$W = (T, D, R, P), \quad (2)$$

where T is a set of tasks, D is a set of data dependencies, R is a set of resource requirements, and P is provenance. In a diffusion editing session, tasks include base generation, semantic annotation, mask refinement, crop selection, local diffusion, deterministic projection, compositing, metric computation, caching, and rollback.

We represent the current image state as a semantic layer graph

$$G = (V, E), \quad (3)$$

where each vertex $v_i \in V$ is a semantic layer and each edge $e_{ij} \in E$ represents spatial, hierarchical, or dependency relationships. A layer is defined as

$$\ell_i = (y_i, \mathbf{M}_i, b_i, d_i, h_i, p_i, c_i), \quad (4)$$

where y_i is the semantic label, $\mathbf{M}_i \in \{0, 1\}^{H \times W}$ is the mask, b_i is the bounding box, d_i is optional depth/order metadata, h_i is edit history, p_i is provenance, and c_i is cache state. Hierarchical edges can express relations such as mouth inside face or object in foreground; dependency edges can express likely co-edit regions such as hand–wrist–sleeve or sky–horizon–reflection.

An edit request is

$$e = (q, a, s, \kappa), \quad (5)$$

where q is the target query, a is the action, s is edit strength, and κ encodes user constraints such as quality preference, latency budget, determinism, or preservation requirement. The workflow must resolve q to one or more layers, estimate the effect of a , choose an execution operator, and produce a new version state.

4.2. Execution operators

We define five operators.

Naive full regeneration recomputes the full image from text or image conditioning. It has high global coherence but weak preservation. Its execution cost is approximately the cost of a complete diffusion pass:

$$T_{\text{naive}} \approx \sum_{t=1}^N C_{\theta}(H, W, t), \quad (6)$$

where C_{θ} is the model cost at resolution $H \times W$.

Semantic local diffusion crops a region around the target mask, runs img2img or inpainting-like diffusion on the crop, and composites the result through the mask:

$$x' = \mathbf{M}_i \odot f_{\theta}(x_{b_i}, e) + (1 - \mathbf{M}_i) \odot x. \quad (7)$$

The expected cost is

$$T_{\text{local}} \approx T_{\text{det}} + \sum_{i=1}^{N_i} C_{\theta}(h_i, w_i, t) + T_{\text{comp}}, \quad (8)$$

where T_{det} is detector time, $h_i \times w_i$ is crop size, N_i is the local denoising step count, and T_{comp} is compositing time.

Cached semantic re-edit reuses the existing semantic mask, bounding box, crop policy, and layer provenance. It removes repeated detector cost:

$$T_{\text{cached}} \approx \sum_{i=1}^{N_i} C_{\theta}(h_i, w_i, t) + T_{\text{comp}}. \quad (9)$$

This operator represents the strongest practical advantage of the workflow model.

Deterministic projection applies a non-diffusion transformation to the layer, such as hue or color projection:

$$x' = \mathbf{M}_i \odot g(x_i, a) + (1 - \mathbf{M}_i) \odot x, \quad (10)$$

where g is deterministic. It is suitable for simple color edits when exact locality and speed matter more than generative realism.

Rollback restores a prior layer or composite state:

$$x' = \text{restore}(v_k, \ell_i), \quad (11)$$

where v_k is a stored version. Rollback uses stored layer state and does not invoke diffusion in the prototype.

4.3. Objective function

The workflow policy chooses an operator o from a candidate set $\mathcal{O}$. The objective below should be read as a cost model for comparing feasible operators, not as a claim that the current prototype learns the weights automatically. The prototype uses explicit rules and measured operator costs; the same state variables can later support an optimizer or learned scheduler. We write the normalized operator score as

$$J(o | e, \ell_i, C) = [\alpha \widehat{T}_o + \beta \widehat{D}_o + \gamma \widehat{Q}_o + \delta \widehat{U}_o + \eta \widehat{R}_o], \quad (12)$$

and the selected operator is

$$\pi(e, \ell_i, C) = \arg \min_{o \in \mathcal{O}_{\text{feasible}}} J(o | e, \ell_i, C). \quad (13)$$

Here $\widehat{T}_o$ is normalized latency, $\widehat{D}_o$ is outside-target drift, $\widehat{Q}_o$ is expected quality or artifact risk, $\widehat{U}_o$ is user effort, and $\widehat{R}_o$ is resource cost. If watt telemetry or cloud billing is available, $\widehat{R}_o$ can be instantiated as normalized energy or monetary cost. In the present benchmark, no power draw is measured, so $\widehat{R}_o$ is instantiated only as a compute-work proxy:

$$\widehat{R}_o = \frac{N_o A_o}{N_{\text{full}} H W} + \chi_o \frac{T_{\text{det}}}{\bar{T}_{\text{det}}}, \quad (14)$$

where N_o is the number of denoising steps, A_o is the rendered pixel area, $H \times W$ is the full image area, and $\chi_o = 1$ when the operator invokes detector inference. This proxy does not claim to be a joule measurement; it records the relative amount of diffusion work and detector work exposed to the scheduler. C denotes context such as available cache, model family, accelerator state, privacy preference, and deadline.

Outside-target drift is estimated as

$$D(x, x', \mathbf{M}) = \frac{1}{|\Omega_{\mathbf{M}}|} \sum_{p \in \Omega_{\mathbf{M}}} \mathbb{I}(\|x_p - x'_p\|_1 > \tau), \quad (15)$$

where $\Omega_{\mathbf{M}}$ is the non-target pixel set and τ is a pixel-change threshold. Inside-target change is

$$I(x, x', \mathbf{M}) = \frac{1}{|\Omega_{\mathbf{M}}|} \sum_{p \in \Omega_{\mathbf{M}}} \|x_p - x'_p\|_1. \quad (16)$$

Boundary risk is measured on a dilated mask band:

$$B = \text{dilate}(\mathbf{M}, r) - \text{erode}(\mathbf{M}, r), \quad (17)$$

and evaluated using the same pixel-change statistic. Boundary-band change is introduced here as a proxy for structural blending and seam artifacts. It is not necessarily bad; it often indicates that color, antialiasing, shadow, or shading changed near the seam. However, high boundary changes can signal visible artifacts and should be interpreted with visual inspection or perceptual metrics.

4.4. Amortized reuse model

The value of semantic annotation depends on session length. A first edit pays detector cost, but repeated edits reuse the same layer state. Let k be the number of edits applied to the same semantic layer in a session. A naive session cost is approximated as

$$T_{\text{naive}}(k) = T_{n,1} + (k - 1)T_{n,2}, \quad (18)$$

where $T_{n,1}$ is the first naive edit and $T_{n,2}$ is a subsequent naive variant. A semantic session cost is

$$T_{\text{sem}}(k) = T_{\text{det}} + T_{s,1} + (k - 1)T_{s,c}, \quad (19)$$

where $T_{s,1}$ is the first local render and $T_{s,c}$ is a cached re-edit. Semantic execution is beneficial when

$$T_{\text{det}} + T_{s,1} + (k - 1)T_{s,c} < T_{n,1} + (k - 1)T_{n,2}. \quad (20)$$

Solving for k gives an approximate break-even condition:

$$k > 1 + \frac{T_{\text{det}} + T_{s,1} - T_{n,1}}{T_{n,2} - T_{s,c}}. \quad (21)$$

Using the measured benchmark means $T_{n,1} = 35.546$ s, $T_{n,2} = 35.741$ s, $T_{\text{det}} = 13.543$ s, $T_{s,1} = 30.132$ s, and $T_{s,c} = 8.115$ s. The semantic path is slower for $k = 1$, but faster already at $k = 2$. This is the mathematical reason that repeated edits and version exploration are the strongest use case.

4.5. Layer dependency risk

Not all local edits are independent. A command such as “change the hand” may affect fingers, wrist, sleeve, object contact, shadow, and reflection. We model this with a dependency expansion function

$$\Gamma(\ell_i, a) \subseteq V, \quad (22)$$

which returns layers that may need protection or co-editing for action a . The effective edit support is then

$$\mathbf{M}_{\text{eff}} = \bigcup_{\ell_j \in \Gamma(\ell_i, a)} w_j \mathbf{M}_j, \quad (23)$$

where w_j can encode soft dependency weights. A concrete graph construction can combine geometry, attention, depth, and contact evidence:

$$s_{ij} = \lambda_m \text{IoU}(\mathbf{M}_i, \mathbf{M}_j) + \lambda_a O(A_i, A_j) + \lambda_z Z(d_i, d_j) + \lambda_c C(\partial \mathbf{M}_i, \partial \mathbf{M}_j), \quad (24)$$

with an edge $(i, j) \in E$ when $s_{ij} > \theta_E$. $O(A_i, A_j)$ measures overlap between token-attention maps, for example from DAAM; $Z(d_i, d_j)$ encodes depth-order or occlusion compatibility; and $C(\partial \mathbf{M}_i, \partial \mathbf{M}_j)$ captures boundary contact or near-contact. The current prototype implements only the geometry part of this idea through crop padding and boundary bands, because the benchmark uses post-generation GroundingDINO+SAM2 masks rather than stored attention maps. The formulation makes the missing dependency edges explicit and shows how a later implementation can build E programmatically instead of relying on a fixed padding heuristic. Dependency expansion changes crop size, latency, and expected quality risk, so it is relevant to any scheduler that uses layer geometry.

4.6. Determinism and reproducibility

Diffusion pipelines are only conditionally deterministic. A run is reproducible when the model checkpoint, VAE, scheduler, timestep sequence, seed, prompt, negative prompt, guidance scale, precision, image size, backend, and software versions are fixed. Even then, hardware kernels and mixed precision can introduce small numerical differences. Our workflow model treats determinism as a provenance property rather than an assumption. Each artifact records enough metadata to make repeated execution auditable, and deterministic projection/rollback are separated from stochastic or numerically sensitive diffusion operators.

5. System Architecture

Figure 1 shows the prototype architecture. A request enters the system as either a base generation prompt or an edit to an existing image. Base generation produces the initial image and provenance state. Semantic annotation then creates named candidate layers using GroundingDINO+SAM2 in the heavy benchmark. The resulting Image DOM stores masks, boxes, labels, crop policies, version identifiers, and provenance. The policy engine selects an execution operator. The execution layer runs full diffusion, local diffusion, deterministic projection, cached re-edit, or rollback. Finally, the compositor writes a new version and logs metrics.

5.1. Semantic Image DOM

The Image DOM is the persistent state that makes repeated edits efficient. For each layer, it stores the mask, bounding box, label, detector backend, detector confidence where available, crop geometry, parent/child relations, and edit history. It also stores global provenance: model checkpoint name, scheduler, inference steps, seed, image size,

guidance scale, and prompt. The representation is intentionally close to workflow provenance models: it records entities, activities, and derivations rather than only final pixels [10].

The current prototype uses file-system persistence for simplicity. Each sample output folder contains the base image, detector mask overlay, naive edit outputs, semantic local outputs, projection-only output, rollback output, sample grid, and a metrics record. This is sufficient for reproducible experiments. A production implementation would replace this with a content-addressed store or workflow database.

5.2. Detector and mask stage

The heavy benchmark uses GroundingDINO for open-vocabulary target localization and SAM2 for mask generation. The target labels are selected from the edit category: lips for face edits, shirt for body edits, mug for object edits, and sky for landscape edits. The detector stage returns a mask and bounding box. The local renderer expands the bounding box with padding and snaps to a crop size supported by the model backend.

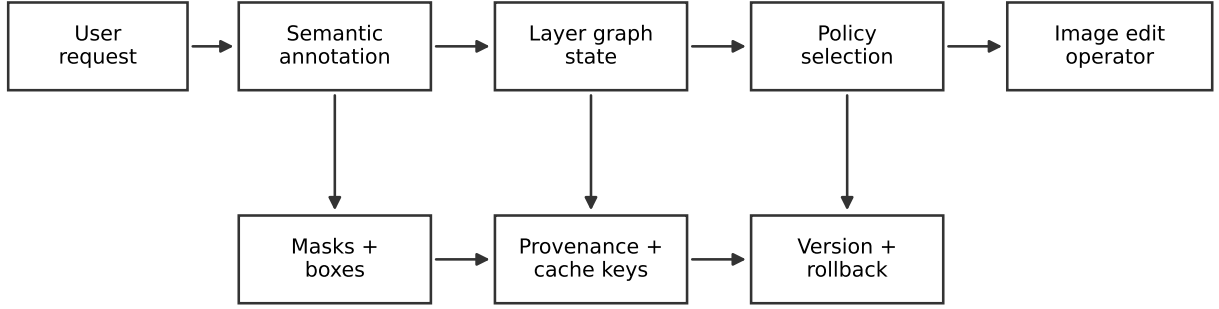
Detector time is included as a separate metric because semantic annotation is expensive enough to change the conclusion. A one-off edit with no cached layer state may be slower than naive generation, while a repeated edit on the same layer avoids detection and becomes much faster.

5.3. Execution policy

The prototype policy is rule-based but grounded in the cost model above. Projection-only is selected for deterministic color changes when the target layer is already known. Semantic local diffusion is selected when the edit requires generative texture or lighting. Cached re-edit is selected when the layer mask and crop state already exist. Naive regeneration is used as the baseline and remains appropriate for global, structural, or highly entangled edits.

Algorithm 1: Rule-based semantic workflow policy.

1. Input edit $e = (q, a, s, \kappa)$, candidate layers V , context C , and version state.
2. Resolve q to a target layer ℓ_i , or run detector inference if no matching layer exists.
3. Construct feasible operators $\mathcal{O}_{\text{feasible}}$: rollback, projection, cached local edit, first local edit, and full regeneration.
4. Remove operators that violate hard constraints, such as unavailable cache, privacy restrictions, unsupported model family, or missing mask.
5. Estimate $\widehat{T}_o$, $\widehat{D}_o$, $\widehat{Q}_o$, $\widehat{U}_o$, and $\widehat{R}_o$ from layer geometry, cache state, detector status, crop size, and benchmarked model family.
6. Prefer rollback for undo, projection for simple deterministic color edits, cached local editing for repeated local variants, first local editing when preservation is required, and full regeneration for global or highly entangled edits.



Semantic state makes masks, provenance, cache reuse, and rollback explicit workflow artifacts.

Figure 1: Semantic layer workflow architecture. The key difference from a monolithic diffusion call is that semantic annotation, layer graph state, cache keys, and version provenance are first-class workflow artifacts.

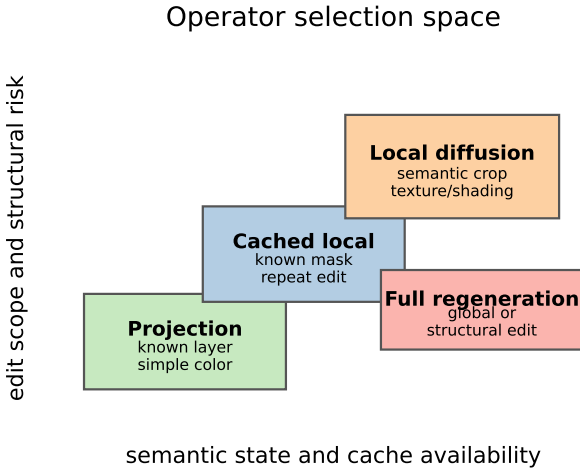


Figure 2: Conceptual operator selection space. Simple local edits favor deterministic projection or cached local rendering, while structurally broad edits may require full regeneration.

7. If more than one operator remains plausible, return the feasible operator with the lowest $J(o | e, \ell_i, C)$.

Figure 2 gives a conceptual view of the operator space. The boundary between regions is not fixed; it depends on detector availability, cache state, latency budget, crop size, and quality requirements. This is why we report both generation-only and detector-inclusive speedups.

5.4. Implementation and artifact layout

The prototype is implemented as Python scripts around local Hugging Face Diffusers pipelines and local model checkpoints [25]. The heavy benchmark driver creates a resumable output directory and appends a JSON

record after every completed sample. This design was important in practice because SDXL model transitions can fragment MPS memory; a resumed process can skip completed records and continue from the next missing model/case/sample tuple.

Each sample directory follows the same structure:

```

sample_XX/
  base.png
  mask_overlay.png
  naive_v1.png
  naive_v2.png
  semantic_raw_v1.png
  semantic_projected_v1.png
  semantic_cached_v2.png
  projection_only.png
  rollback.png
  sample_grid.png
  metrics.json
  
```

The exact filenames vary slightly by operator, but the invariant is that every image artifact is paired with a metrics record. The aggregate directory contains a full JSON file, a Markdown report, a model-case CSV matrix, and representative sample grids. This layout is intentionally simple: it can be inspected without a database, archived with the paper, or converted into a formal workflow provenance store.

The policy engine is therefore a transparent heuristic scheduler rather than an optimized solver. This is a deliberate prototype choice: the experiment first tests whether the Image DOM exposes useful scheduling features. The parameters $\alpha, \beta, \gamma, \delta, \eta$ can later be fitted from user preferences, measured energy, or deployment constraints, but the current paper reports the rule-based policy and the cost-model simulations separately.

5.5. Failure handling

Long-running generative workflows benefit from partial-failure tolerance. The benchmark treats each sample tuple as an idempotent unit identified by model key, case key, and sample index. If execution stops after writing a record, the next run skips that tuple. This allowed the local benchmark to retain progress after memory-related restarts.

6. Experimental Methodology

6.1. Research questions

The evaluation is organized around eight questions:

RQ1 Does semantic workflow execution reduce non-target drift compared with naive full-image regeneration?

RQ2 When detector overhead is excluded, does local semantic rendering reduce generation time?

RQ3 What is the first-edit latency when semantic detection is included?

RQ4 How much speedup is obtained from cached semantic re-edits and versioning?

RQ5 How does performance vary by model family and edit category?

RQ6 What trade-offs appear between deterministic projection, local diffusion, and full regeneration?

RQ7 How sensitive are scheduling conclusions to heterogeneous resource assumptions?

RQ8 Do perceptual and dependency-risk audits support the pixel-level preservation results?

6.2. Benchmark design

We ran a paired benchmark over 10 compatible local diffusion checkpoints, four edit categories, and five seed variants per category, for 200 paired samples. Each sample executes the same workflow pattern:

1. Generate or load a base image at 768×768 .
2. Execute a naive full-image first edit.
3. Execute a second naive variant edit.
4. Detect the target semantic layer using GroundingDINO and SAM2.
5. Execute projection-only deterministic editing.
6. Execute semantic local diffusion on the target crop and composite through the mask.
7. Execute a cached semantic re-edit using the same layer state.

8. Execute rollback to a prior version.

9. Save images, masks, sample grids, and metrics.

All runs use 20 denoising steps. The four cases are face/lips color editing, human body/shirt editing, object/mug editing, and landscape/sky editing. These categories were chosen because they cover small facial parts, medium human regions, discrete objects, and broader background regions. The local rendering crop is determined from the SAM2 mask and may be smaller than the full image, but it is padded and quantized for model compatibility.

The benchmark ran on the local workstation using the available Apple MPS backend. Some SDXL model transitions required process restart because of MPS memory fragmentation; the benchmark is resumable and persisted completed records were reused rather than recomputed. This is reported as part of the realistic local execution environment, not hidden as a failure. The final benchmark completed 200 samples with zero per-sample errors. Four incompatible local checkpoint files were excluded because they were inpainting-only, SD3, non-image-generation, or incomplete checkpoint entries.

To reduce the hardware-specificity threat, we also ran measured CUDA experiments on Google Cloud Platform. The main cloud run uses a *g2-standard-8* VM with one NVIDIA L4 GPU and Juggernaut X RunDiffusion from the project model bucket, corresponding to the RunDiffusion Juggernaut X v10 model-card family [26]. It reruns the detector-inclusive workflow for 100 samples across the same four edit categories. A second L4 experiment sweeps denoising steps {10, 20, 30} and crop sizes {256, 384, 512, 640, 768}, yielding 45 operator timing samples. A compact NVIDIA T4 tier repeats the 20-step operator measurement with CPU offload and VAE tiling. Finally, a long-session L4 benchmark runs repeated edit sessions across Juggernaut X, SDXL Base 1.0, DreamShaper XL, DreamShaper 8, and OpenJourney v4. Each session first annotates the target once and then runs enough full-image and semantic local variants to produce 5–10 minutes of naive generation work. These cloud experiments are still not a full multi-tier scheduler, but they provide measured accelerator evidence rather than relying only on scaled local timings.

6.3. Models

Table 1 lists the 10 compatible checkpoints. The set intentionally includes both SD1.5-family and SDXL-family models because throughput and img2img cost differ substantially across families. The table reports the benchmark names used in the local output records; the corresponding public Hugging Face model-card identifiers are cited below the table where available.

The baseline Stable Diffusion checkpoints correspond to the Stable Diffusion v1.5 and SDXL Base 1.0 model

Table 1: Evaluated diffusion checkpoints. Each model is evaluated on four edit categories with five seed variants per category.

Model	Family	Samples	Naive (s)	Semantic local (s)	Detector (s)
Deliberate v6	SD1.5	20	12.213	8.180	12.954
DreamShaper 8	SD1.5	20	31.388	8.298	13.292
DreamShaper XL 1.0	SDXL	20	39.873	55.469	14.677
Juggernaut Reborn	SD1.5	20	11.390	4.973	12.778
Juggernaut XL v2	SDXL	20	118.220	75.622	15.298
OpenJourney v4	SD1.5	20	23.941	7.496	12.384
RealVisXL V4.0	SDXL	20	53.656	62.119	14.488
Realistic Vision V6.0 B1	SD1.5	20	10.540	9.139	12.194
SDXL Base 1.0	SDXL	20	43.568	60.456	14.151
Stable Diffusion 1.5 Base	SD1.5	20	10.675	9.567	13.220

cards [27, 28]. The remaining rows cite the public model-card families used for provenance: DreamShaper [29, 30], OpenJourney v4 [31], Realistic Vision and RealVisXL [32, 33], Deliberate [34], Juggernaut Reborn [35], and RunDiffusion Juggernaut XL [36]. Since third-party model cards and licenses can change, the artifact records local checkpoint filenames and output metrics but does not redistribute weights.

6.4. Metrics

The benchmark records four groups of metrics.

Latency metrics include naive first-edit time, naive second-edit time, detector time, semantic local render time, cached semantic re-edit time, projection-only time, and rollback time. We report three speedup variants:

$$S_{\text{gen}} = T_{\text{naive}}/T_{\text{semantic}}, \quad (25)$$

$$S_{\text{incl}} = T_{\text{naive}}/(T_{\text{det}} + T_{\text{semantic}}), \quad (26)$$

$$S_{\text{cached}} = T_{\text{naive}2}/T_{\text{cached}}. \quad (27)$$

S_{gen} asks whether local rendering is faster than full rendering once a layer is known. S_{incl} asks whether the first semantic edit is faster for a user with no cache. S_{cached} asks whether repeated edits benefit from the persisted Image DOM.

Locality metrics measure outside-target changed pixels, inside-target mean absolute difference, and boundary-band changed pixels. These quantify preservation and seam risk. The outside-target metric is especially important because full-image regeneration can be visually plausible while still destroying identity or layout.

Layer geometry metrics include mask area percentage, bounding-box area percentage, crop size, and crop-pixel percentage. These help explain why some local edits are faster than others.

Perceptual metrics include LPIPS full-image, outside-mask, and target-crop distances computed between the base image and each output variant. LPIPS is not a semantic-intent metric, but it tests whether the low-level preservation results also hold under a learned perceptual distance.

Dependency-contact metrics approximate layer dependency risk with a ring immediately outside the target mask.

The metric measures how much the area most likely to contain neighboring structures changes after semantic compositing. It is a geometry-only proxy, not a replacement for attention-map or depth-based dependency edges.

Versioning metrics include cached re-edit time and rollback time. These directly test whether semantic state makes repeated edits and undo operations cheap.

Resource proxy metrics include crop-pixel percentage and detector invocation. These are not watt-level energy measurements. They instantiate the resource term $\widehat{R}_o$ as relative diffusion area and detector work so that the scheduler can compare full rendering, local rendering, projection, and rollback on a common scale.

6.5. Reproducibility artifacts

The benchmark stores machine-readable results in JSON and CSV. The JSON contains all per-sample records and aggregate summaries. The CSV contains the model-by-case matrix used for compact comparison. Figures in this manuscript are generated from those files rather than manually transcribed. The manuscript directory also includes copied sample grids for the appendix. The most important reproducibility fields are model key, model family, case key, sample index, seed, image width/height, steps, mask area, bounding-box area, crop size, detector time, render times, speedups, drift metrics, and artifact directory.

The public artifact does not include model checkpoints or detector weights because those files are large and governed by separate licenses. It includes the scripts, reports, aggregate metrics, model-case matrix, manuscript source, and representative image artifacts needed to inspect the reported results.

7. Results

7.1. Overall performance

Table 2 summarizes the full 200-sample benchmark. Semantic local rendering excluding detector cost is faster than naive full-image generation on average. However, detector-inclusive first edits are slower on average. Cached re-edits are the strongest performance result, with a 7.041x

Table 2: Overall benchmark results across 200 paired executions.

Metric	Mean
Naive full edit time	35.546 s
Semantic local edit time, detector excluded	30.132 s
Detector time	13.543 s
Projection-only edit time	0.031 s
Generation-only semantic speedup	1.788x
One-off detector-inclusive speedup	0.809x
Cached version re-edit speedup	7.041x
Projection-only speedup	1227.276x
Rollback time	0.188 s
Naive outside-target changed pixels	93.954%
Semantic outside-target changed pixels	0.000%
Semantic boundary-band changed pixels	42.301%
Mean crop-pixel work proxy	42.145%
Mean mask area	6.032%

Table 3: Operator-level baseline decomposition across 200 samples.

Operator	Mean time
Naive full regeneration	35.546 s
Detector + local semantic edit	43.675 s
Known-mask local rendering	30.132 s
Cached semantic re-edit	8.115 s
Deterministic projection	0.031 s
Rollback	0.188 s

average speedup. Projection-only edits are effectively instantaneous for simple color changes, and rollback is near-interactive.

The detector-inclusive number is the first-edit result. When a user edits a region for the first time and the system has no cached semantic layer, GroundingDINO+SAM2 adds substantial overhead. This overhead can be justified when preservation matters, but it does not guarantee lower latency. Once the semantic layer exists, repeated variants avoid detection and reuse the layer state.

7.2. Baseline decomposition

Table 3 separates the benchmark into operator-level baselines. This comparison is important because the workflow is not only evaluated against full-image regeneration. The detector-inclusive local edit represents an ordinary first local edit with semantic detection but no reuse. Known-mask local rendering removes detector cost. Cached re-editing then measures the additional value of persisted workflow state.

The decomposition shows why caching is central to the contribution. The first detector-inclusive local edit is slower than full regeneration on average, but known-mask local rendering is faster, and cached semantic re-editing is 5.382x faster than the detector-inclusive local baseline. The workflow gain therefore comes from making semantic state persistent and reusable, not from assuming that every local edit is automatically faster than a full-image edit.

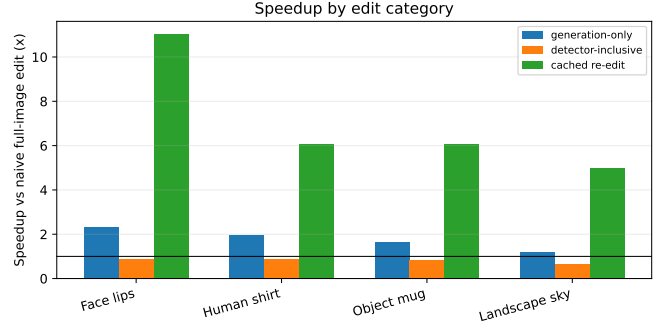


Figure 3: Speedup by edit category. Cached semantic re-edit is positive for all categories, while detector-inclusive first-edit speedup is below 1x on average.

7.3. Edit category effects

Table 4 and Figure 3 show results by edit category. Face/lips edits have the highest cached speedup at 11.055x because the editable region is small and repeated color changes are cheap. Human body/shirt and object/mug edits show similar cached speedups near 6x. Landscape/sky edits are less favorable because the semantic region and crop are often larger, but cached re-editing still averages 4.966x.

These results support a policy-based interpretation. If the target is small and the edit is simple, projection-only or cached local editing is attractive. If the target is broad, first-edit performance may not improve, but preservation can still justify semantic execution. Layer geometry and cache state are therefore useful scheduling features.

7.4. Model effects

Figure 4 shows strong model dependence. DreamShaper 8, OpenJourney v4, and Juggernaut XL v2 achieve detector-inclusive first-edit speedups above 1x. Other models, especially several SDXL checkpoints, are slower in the semantic first-edit path. This occurs because local img2img throughput and full-generation throughput differ by checkpoint and pipeline implementation. It would be misleading to claim that local semantic rendering is always faster. To make the non-SDXL evidence explicit, Table 5 reports the five strongest models ranked by cached semantic re-edit speedup from the completed 200-sample benchmark. Four of these five are SD1.5-family checkpoints.

The workflow conclusion remains stable across models: cached re-editing is positive for every checkpoint. The weakest cached result is RealVisXL V4.0 at 3.172x, while the strongest is OpenJourney v4 at 12.229x. The top-five table also shows why the policy cannot use model family alone: DreamShaper 8 and OpenJourney v4 are strong detector-inclusive first-edit cases, while Juggernaut Reborn and Deliberate v6 are faster mainly when detector state is reused. This suggests that semantic caching is a robust systems mechanism even when first-edit acceleration is model-dependent.

Table 4: Results by edit category. Each category contains 50 paired samples.

Case	Naive	Local	Det.	Gen.	Incl.	Cached	Naive drift	Sem. drift
Face lips	35.796	28.063	13.871	2.313x	0.877x	11.055x	90.355%	0.000%
Human body shirt	38.372	30.619	13.517	1.958x	0.886x	6.085x	91.908%	0.000%
Landscape sky	31.858	31.358	13.636	1.211x	0.634x	4.966x	98.745%	0.000%
Object mug	36.160	30.488	13.151	1.669x	0.838x	6.058x	94.809%	0.000%

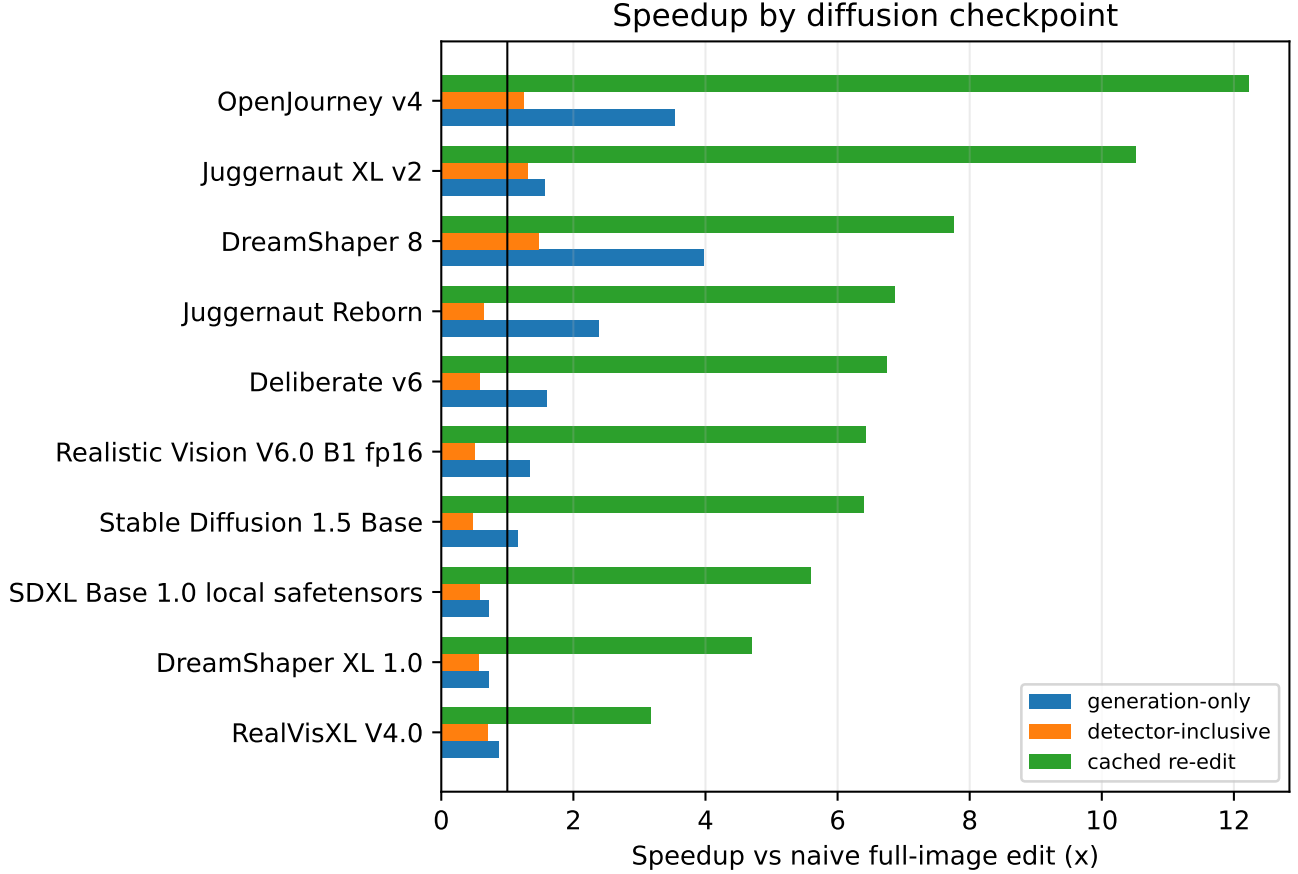


Figure 4: Speedup by model. The generation-only metric isolates local rendering cost; the detector-inclusive metric represents first-edit user latency; cached re-edit represents repeated semantic layer editing.

7.5. Locality and drift

Naive full-image regeneration changes an average of 93.954% of outside-target pixels. Semantic compositing changes 0.000% outside-target pixels under the exact pixel-change metric because the edited crop is composited through the semantic mask and the unmasked base image is preserved.

This metric is intentionally strict. It does not claim that semantic edits are always perceptually superior; it shows that the workflow preserves non-target pixels by construction. This is useful when identity, layout, text, object positions, or scientific visual state must remain stable. The boundary-band change average of 42.301% indicates that changes occur near seams, so visual seam quality still requires inspection and better blending methods.

7.6. Trade-offs

The main trade-off is between first-edit latency and state reuse. Semantic editing pays an annotation cost before it can exploit locality. In the benchmark, this cost is large enough that the first detector-inclusive edit is slower than naive regeneration on average. This is acceptable when the user expects to make several changes to the same region, compare variants, or preserve the surrounding image exactly. It is less attractive for a single low-stakes edit where global regeneration is visually acceptable and latency is the only concern.

The second trade-off is between preservation and visual integration. Mask compositing keeps non-target pixels fixed, which is useful for identity, layout, text, product images, or scientific visual state. The same constraint can make boundaries more visible if the local edit changes lighting, texture, or geometry near the mask edge. In those

Table 5: Top-five cross-model results from the 200-sample local heavy benchmark, ranked by cached semantic re-edit speedup. Each row aggregates 20 paired executions: four edit categories with five seeds each.

Rank	Model	Family	Naive	Local	Detector	Cached	Gen.	Incl. / cached
1	OpenJourney v4	SD1.5	23.941	7.496	12.384	3.618	3.537x	1.246x / 12.229x
2	Juggernaut XL v2	SDXL	118.220	75.622	15.298	26.529	1.567x	1.303x / 10.514x
3	DreamShaper 8	SD1.5	31.388	8.298	13.292	4.883	3.970x	1.483x / 7.762x
4	Juggernaut Reborn	SD1.5	11.390	4.973	12.778	2.109	2.392x	0.643x / 6.872x
5	Deliberate v6	SD1.5	12.213	8.180	12.954	2.610	1.601x	0.579x / 6.744x

cases, semantic execution is still useful as a controlled editing mode, but it should be paired with better blending, boundary-aware scoring, or a policy that expands the crop when the edit is structurally coupled to nearby regions.

A third trade-off is implementation complexity. Maintaining masks, cache keys, provenance, and version state is more work than calling a diffusion pipeline once. The benefit appears when edits are iterative, audited, or reused: cached re-edits become faster, rollback is cheap, and the system can explain which model, mask, crop, and parameters produced each result. For casual one-shot generation this overhead may not be justified; for repeated editing workflows it becomes the mechanism that makes locality and reproducibility practical.

7.7. Versioning and rollback

Rollback averages 0.188 s. This supports the versioning model: undoing a semantic edit restores cached state rather than invoking diffusion. In interactive workflows, rollback changes the editing process from destructive regeneration to a versioned exploration tree. Users can compare variants of the same object while preserving the rest of the image and retaining intermediate states.

The cached re-edit result is similarly important. A first semantic edit may be slower because of detector overhead, but subsequent edits amortize semantic annotation. This is analogous to scientific workflows where expensive preprocessing or indexing steps become worthwhile when reused across many downstream tasks.

7.8. Scheduling-policy simulation

Table 6 applies a simple scheduling simulation to the 200 measured records. For a session with k edits on the same layer, the naive policy uses full regeneration for every variant, the semantic policy pays detector cost once and then uses cached re-edits, and the oracle latency policy chooses the faster of the two measured paths per sample. This is not a new deployment experiment, but it evaluates the scheduling decision that the workflow representation exposes.

The policy result is stronger than the mean break-even calculation because it shows how often each decision is preferred across heterogeneous models and edit categories. For $k = 1$, semantic editing is not a latency default. For $k = 2$, reuse already changes the decision for most samples. For $k = 5$, semantic execution is faster almost everywhere. This supports the systems claim that session

Table 6: Measured scheduling-policy simulation. k is the number of edits on the same semantic layer. Times are mean seconds across 200 samples.

k	Naive	Semantic	Oracle	Sem. wins
1	35.546	43.675	30.780	25.5%
2	71.287	51.791	49.543	60.0%
3	107.027	59.906	59.839	98.0%
5	178.509	76.137	75.967	99.5%

context, cache state, and preservation constraints should be explicit scheduler inputs rather than hidden inside a single image-generation call.

Table 7 further separates latency-only scheduling from preservation-aware scheduling. The latency-only policy selects semantic execution only when it is faster for the current sample and session length. The preservation-first policy always selects semantic execution when a target mask exists, because outside-target stability is treated as a hard requirement. The balanced policy uses latency when $k = 1$, but selects semantic execution for repeated edits because cache reuse and outside preservation dominate after the first edit. The oracle column is the measured faster path and is included only as a latency reference.

The ablation exposes the role of the objective weights without claiming that the prototype learns them. If latency is the only objective, semantic execution is not a universal first-edit choice. If preservation is a hard constraint, the semantic path is chosen even when it is slower. If the session contains repeated edits, the balanced rule converges to semantic execution because the detector cost is amortized and the non-target preservation gain is large. This bridges the cost model and the implemented rule engine: the current system evaluates a transparent finite set of policies rather than solving a learned optimization problem.

7.9. Heterogeneous resource sensitivity

The measured benchmark uses Apple MPS, so the absolute detector and diffusion times should not be treated as universal. To test whether the scheduling conclusion depends only on this desktop configuration, Table 8 applies a sensitivity model to the 200 measured records. The model scales detector time and diffusion time separately:

$$T_{\text{naive}}^r = s_{\text{diff}}^r T_{\text{naive}} + L^r, \quad (28)$$

$$T_{\text{sem}}^r = s_{\text{det}}^r T_{\text{det}} + s_{\text{diff}}^r T_{\text{local}} + L^r, \quad (29)$$

$$T_{\text{cached}}^r = s_{\text{diff}}^r T_{\text{cached}} + L^r, \quad (30)$$

Table 7: Policy ablation over the 200 measured records. Latency is mean seconds for the k -edit session; drift is mean outside-target drift.

k	Policy	Semantic selected	Latency	Outside drift
1	Latency-only / oracle	25.5%	30.780	69.973%
1	Preservation-first	100.0%	43.675	0.000%
1	Balanced	25.5%	30.780	69.973%
2	Latency-only / oracle	60.0%	49.543	38.102%
2	Preservation-first	100.0%	51.791	0.000%
2	Balanced	100.0%	51.791	0.000%
3	Latency-only / oracle	98.0%	59.839	1.773%
3	Preservation-first	100.0%	59.906	0.000%
3	Balanced	100.0%	59.906	0.000%
5	Latency-only / oracle	99.5%	75.967	0.455%
5	Preservation-first	100.0%	76.137	0.000%
5	Balanced	100.0%	76.137	0.000%

where s_{det}^r and s_{diff}^r are resource-tier scaling factors and L' is a transfer or orchestration penalty. The factors are not presented as measurements of named commercial hardware; they are stress-test scenarios that represent slower edge execution, the measured local run, faster CUDA-class execution, and a cloud accelerator with transfer overhead.

The projection preserves the main conclusion but changes its magnitude. Faster accelerators reduce both full-generation and local-diffusion latency, while faster detectors narrow the first-edit penalty. In all four scenarios, the one-off semantic path remains slower on average, but the three-edit session is faster because detection is amortized and cached re-edits are cheap. This is the compute-continuum claim supported by the present evidence: the exact break-even point is resource dependent, but the Image DOM exposes the variables a placement scheduler needs to recompute the decision.

Table 9 reports the measured detector-inclusive CUDA tier. Unlike the earlier operator-only run, this experiment executes GroundingDINO+SAM2 on the L4 for every sample. The detector is loaded once, and warm per-sample detector inference averages 0.235 s. Full-image naive editing averages 4.882 s, local semantic diffusion averages 0.939 s, and the detector-inclusive semantic path averages 1.174 s. The resulting detector-inclusive speedup is 4.28x, while cached re-edits average 5.52x. This directly changes the interpretation of the local Apple MPS result: the first-edit penalty is not inherent to the workflow model, but depends on the relative speed of detector inference and diffusion on the chosen resource tier.

Table 10 reports the larger L4 operator sweep. It shows the expected dependency between crop area and speedup: small and medium crops benefit most, while a 768×768 img2img pass is still faster than full txt2img in this configuration but no longer represents a locality win. Across 45 timing samples, mean operator speedup is 4.42x and median speedup is 5.08x. At the paper’s 20-step setting, 384×384 and 512×512 crops achieve 6.18x and 5.17x speedup, respectively.

Table 11 adds a compact T4 tier. The T4 required CPU offload and VAE tiling to avoid out-of-memory failures with both SDXL pipelines. Under that constrained execution mode, the 20-step full-image edit averages 16.658 s and local crops average 7.882 s, giving 2.21x mean speedup.

This is slower than the L4 tier, but it is still useful: it shows that memory capacity and pipeline residency are scheduling variables, not incidental implementation details.

Table 12 reports the long-session L4 benchmark. Unlike the short per-image timing runs, this experiment measures repeated user-like edit sessions: one target annotation followed by many variant edits on the same semantic layer. The benchmark covers five model families and two target types. It executes 303 requested variants, totaling 2786.7 s of naive full-image regeneration and 528.9 s of detector-inclusive semantic execution. Mean detector-inclusive session speedup is 5.37x, with a range from 4.87x to 6.03x. The result strengthens the amortization claim: when the editing task is an exploratory session rather than a single isolated call, semantic state converts detector and mask cost into a small fixed overhead.

The project bucket also contains Flux, Wan, Qwen, Z-image, GGUF, and ComfyUI-specific weights. A runtime audit found 189 model objects: one directly measured SDXL checkpoint in the paper runtime, 52 objects requiring a separate runtime family, five GGUF objects that are not comparable under the SDXL Diffusers path, and 131 non-checkpoint or manually triaged objects. For this reason, the paper reports the bucket audit as a reproducibility and scope result rather than claiming a direct multi-model parity benchmark across incompatible runtimes.

Table 13 adds a break-even stress test. Under the measured Apple MPS profile, the median break-even session length is $k = 1.79$, and 98.0% of samples prefer semantic execution by $k = 3$. The result remains strong for CUDA-class and cloud-accelerator scenarios. In the adverse slow-detector/fast-diffusion case, median break-even increases to $k = 5.55$, and only 24.0% of three-edit sessions prefer semantic execution. This defines where the policy should avoid local semantic execution unless preservation, provenance, or privacy dominates latency.

7.10. Interpreting boundary-band change

The average semantic boundary-band change is 42.301%. This is not a failure rate: the band lies near the mask edge, where color, lighting, and antialiasing changes are expected after compositing. The benchmark treats boundary change as a seam-risk proxy for comparison and triage, not as a perceptual quality score.

Table 8: Heterogeneous resource sensitivity projection using the 200 measured records. Times are projected mean seconds. $k = 3$ denotes three edits on the same semantic layer.

Scenario	s_{det}	s_{diff}	Naive $k = 1$	Semantic $k = 1$	Semantic speed	$k = 3$ speed
Edge-constrained	1.80	1.50	53.320	69.576	0.766x	1.709x
Measured Apple MPS	1.00	1.00	35.546	43.675	0.814x	1.787x
CUDA workstation	0.35	0.45	15.996	18.300	0.874x	1.881x
Cloud accelerator, $L = 1.5$ s	0.15	0.20	8.609	9.558	0.901x	1.639x

Table 9: Measured detector-inclusive SDXL workflow benchmark on GCP NVIDIA L4. The run uses 100 samples across four edit categories at 20 denoising steps.

Metric	Value
Samples completed	100
Naive full-image edit	4.882 s
Detector inference	0.235 s
Semantic local diffusion	0.939 s
Detector-inclusive semantic edit	1.174 s
Cached local re-edit	0.931 s
Generation-only speedup	5.45x
Detector-inclusive speedup	4.28x
Cached speedup	5.52x
Naive outside-mask drift	85.009%
Semantic outside-mask drift	0.000%

Table 10: GCP NVIDIA L4 crop/step operator sweep. Each condition has three repeats. Area is relative to a 768×768 full image.

Steps	Crop	Area	Full	Local / speedup
10	256	11.1%	2.662	0.461 / 5.82x
10	384	25.0%	2.662	0.535 / 4.99x
10	512	44.4%	2.662	0.681 / 3.91x
20	256	11.1%	4.988	0.722 / 6.91x
20	384	25.0%	4.988	0.807 / 6.18x
20	512	44.4%	4.988	0.965 / 5.17x
20	640	69.4%	4.988	1.539 / 3.24x
30	384	25.0%	7.074	1.183 / 5.98x
30	512	44.4%	7.074	1.360 / 5.20x

7.11. Automatic quality-risk audit

Table 14 adds an automatic quality-risk audit over the 200 samples. It does not replace human evaluation; it checks target confidence, edit activity, and seam risk using detector score, inside-target change, and boundary-band change.

Most runs produced measurable target-region changes; a smaller subset should be inspected for weak detection or seam risk. Detector confidence and pixel-change metrics cannot prove user-intent satisfaction, but they provide reproducible triage for human or model-assisted perceptual scoring.

7.12. Perceptual validation layer

Pixel metrics are insufficient for final quality assessment. The evaluation therefore separates systems metrics from perceptual validation. As an immediate check, we compute full-image, outside-mask, and target-crop LPIPS between the base image and all 800 output variants. Table 15 reports the aggregate result.

The cloud detector-inclusive run was also scored with LPIPS (Table 16). Because all cloud samples use the

Table 11: Compact GCP NVIDIA T4 operator tier with CPU offload and VAE tiling. Each condition has two repeats at 20 denoising steps.

Crop	Area	Full	Local	Speedup
384	25.0%	16.658	7.929	2.26x
512	44.4%	16.658	6.526	2.55x
768	100.0%	16.658	9.192	1.81x
All samples	–	16.658	7.882	2.21x

Table 12: Long GCP NVIDIA L4 edit-session benchmark. Each row runs one target annotation followed by repeated variants until the naive full-image session lasts about 5–10 minutes.

Model	Case	Variants	Naive	Semantic	Speedup
Juggernaut X	Mug	45	641.4	131.0	4.90x
Juggernaut X	Shirt	44	628.9	120.4	5.22x
SDXL Base 1.0	Mug	21	300.2	61.7	4.87x
DreamShaper XL	Mug	21	300.5	54.1	5.55x
DreamShaper 8	Mug	58	305.6	54.4	5.62x
DreamShaper 8	Shirt	58	305.8	50.7	6.03x
OpenJourney v4	Mug	56	304.3	56.5	5.38x
All sessions	–	303	2786.7	528.9	5.37x

same SDXL checkpoint and compositing path, the locality pattern is stronger: naive regeneration has outside-mask LPIPS 0.4801, while semantic and cached outputs remain near zero at 0.00019. Target-crop scores remain non-zero, as expected for local edits.

Together, the LPIPS audits support the locality result with a learned perceptual metric. Naive regeneration has high full-image and outside-mask distance because the whole image is resampled; semantic, cached, and projection outputs keep outside-mask LPIPS near zero through compositing. LPIPS does not prove semantic success, but shows that preservation is not an artifact of a single pixel threshold.

Table 17 defines the remaining validation layer for a production scheduler or follow-up benchmark; CLIP/BLIP alignment and human ratings are future semantic-quality checks.

These metrics would change $\widehat{Q}_o$: low outside drift with poor CLIP alignment should not be considered successful, and visible seams should trigger a larger crop, blending, or full regeneration. The pixel audit is a cheap screen; semantic alignment and realism require model-assisted scoring or human labels.

7.13. Dependency-risk audit

The dependency formulation is broader than the implementation: a complete layer graph should use mask

Table 13: Break-even stress test across resource assumptions. $k = n$ columns report the percentage of samples where semantic execution is faster by session length n .

Scenario	Median k^*	$k = 1$	$k = 3$	$k = 5$
Edge-constrained	2.02	24.0%	92.5%	99.5%
Measured Apple MPS	1.79	25.5%	98.0%	99.5%
CUDA-class	1.61	26.5%	99.5%	99.5%
Cloud accelerator	1.58	26.5%	99.5%	99.5%
Fast detector, slow diffusion	0.89	54.0%	100.0%	99.5%
Slow detector, fast diffusion	5.55	0.0%	24.0%	46.0%

Table 14: Automatic quality-risk audit across 200 samples. Thresholds are used only as triage flags.

Metric	Mean	Median	Flagged
Detector selection score	0.456	0.473	10.5% below 0.25
Inside-target change	34.435	34.459	3.5% below 20
Boundary-band change	42.301	42.825	3.0% above 50

overlap, attention overlap, depth ordering, and contact evidence. Because the prototype cannot reconstruct DAAM maps after generation, we add a geometry-only audit. For each sample, a contact ring outside the target mask approximates where neighboring structures would reveal an over-tight crop or poor dependency expansion.

The audit does not solve dependency estimation, but makes the approximation measurable. Mean contact-ring change is 7.172%, far below boundary-band change because the ring excludes the target seam. No sample exceeds the 0.7 high-risk threshold. This suggests that crop and mask compositing usually protect immediate neighbors, while stronger dependency edges are still needed for semantically coupled objects and resource-placement decisions.

7.14. Implications for scheduling

The results suggest three scheduling rules, formalized in Algorithm 1. If no cache exists and latency is the only objective, the scheduler needs detector-inclusive cost. If preservation or reproducibility matters, semantic editing may be selected even when slower. If a layer is edited more than once, detector cost is amortized rapidly.

In a compute-continuum deployment, detector-heavy first edits could use faster accelerators when privacy and cost allow, cached projection or rollback can remain local, and large SDXL diffusion can be offloaded under memory pressure. The Image DOM exposes the semantic scope and cache state needed for such placement.

8. Discussion

8.1. What the system is good at

The benchmark identifies four strong use cases: semantic layers preserve non-target pixels under the implemented compositing metric; cached layer state accelerates repeated edits; deterministic projection is extremely fast for simple color changes; and versioning/rollback are cheap

Table 15: LPIPS perceptual-distance audit over 800 output images. Lower outside-mask LPIPS indicates better perceptual preservation outside the target mask.

Variant	Full image	Outside mask	Target crop
Naive regeneration	0.5848	0.5614	0.7062
Semantic local	0.0364	0.0005	0.2847
Cached semantic	0.0283	0.0005	0.2244
Projection-only	0.0339	0.0004	0.2403

Table 16: Cloud LPIPS audit over the 100-sample GCP L4 detector-inclusive run. The table reports 300 variant comparisons: naive, semantic, and cached outputs for each sample.

Variant	Full image	Outside mask	Target crop
Naive regeneration	0.5266	0.4801	0.6714
Semantic local	0.0085	0.0002	0.0895
Cached semantic	0.0071	0.0002	0.0802

because they are workflow state rather than new generation tasks.

The system is therefore a workflow optimization layer, not a replacement for diffusion editors. It invokes diffusion for realism, deterministic math for local speed, and rollback for state restoration.

8.2. What the system is not yet good at

The weakest result is detector-inclusive first-edit latency. GroundingDINO+SAM2 adds 13.543 s on average locally, so naive regeneration may be faster for one-off edits that do not need preservation. Some SDXL checkpoints also make local img2img slower than full naive generation.

Mask correctness is another limitation. Preservation is measured relative to detector masks, not hand-labeled ground truth; stronger semantic claims require hand-labeled masks or a smaller expert-labeled validation set.

8.3. Compute-continuum implications

The prototype benchmark is not a deployed multi-tier continuum experiment. Table 8 projects measured operator traces under detector, diffusion, and transfer scaling assumptions. The model still informs scheduling: projection and rollback can run locally, detector inference can move when privacy permits, and SDXL diffusion can be offloaded under memory pressure. These are sensitivity results, not measured deployment results.

The policy objective can include resource cost and deadline:

$$(o^*, r^*) = \arg \min_{o,r} \alpha \widehat{T}_{o,r} + \beta \widehat{D}_o + \gamma \widehat{Q}_o + \eta \widehat{R}_{o,r} + \zeta \widehat{M}_{o,r}, \quad (31)$$

where r is a resource tier and $\widehat{M}_{o,r}$ is normalized memory risk. This turns semantic editing into a resource-aware scheduling problem [24]. The layer graph exposes crop size, cache presence, detector availability, and dependency risk.

Table 17: Perceptual validation layer for semantic editing quality.

Metric	Tests	Policy use
CLIP/BLIP align-ment	prompt/edit intent	semantic success risk
LPIPS target distance	perceptual change	over/under-edit risk
LPIPS outside mask	perceptual preservation	identity/layout risk
Human seam rating	boundary artifacts	expand crop/blend
Human task rating	edit satisfaction	policy training label

Table 18: Geometry-only dependency-risk audit over 200 samples. Contact change is measured in the ring just outside the semantic target mask.

Case	Contact change	Risk score	High risk
Face/lips	7.226%	0.467	0.0%
Human body/shirt	6.430%	0.445	0.0%
Landscape/sky	8.270%	0.499	0.0%
Object/mug	6.760%	0.449	0.0%
All samples	7.172%	0.465	0.0%

8.4. FAIR and reproducibility

The benchmark stores seeds, model identifiers, prompts, detector outputs, masks, images, metrics, and reports. A stronger FAIR package would add a container, model hashes, lockfiles, and a public reproducibility bundle; the workflow model already treats provenance as core state.

9. Research Agenda Toward a Compute-Continuum Workflow System

The prototype supports the semantic workflow claim while showing what remains for a full compute-continuum system.

9.1. Latent trajectory reuse

The current implementation caches image-space layers, masks, crops, and versions, but not the full diffusion latent trajectory. A stronger system would store

$$\mathcal{T}^0 = \{z_N^0, z_{N-1}^0, \dots, z_0^0\}, \quad (32)$$

where z_t^0 is the base latent at denoising step t . An edit could restart from an intermediate timestep t_r and constrain the edited trajectory by the semantic mask:

$$z_{t-1}^e = \mathbf{M}^z \odot S_t(z_t^e, c_e) + (1 - \mathbf{M}^z) \odot z_{t-1}^0, \quad (33)$$

where $\mathbf{M}^z$ is the latent-resolution mask and c_e is edit conditioning. This preserves the original trajectory outside the edit support while the target denoises under new conditioning.

This is not whole-step reuse: diffusion steps are global, so changing one region can affect predictions elsewhere. The valid reuse object is the cached latent field with regional projection. The policy would choose t_r from edit strength, dependency risk, and quality target:

$$t_r = \rho(a, s, \Gamma(\ell_i, a), q_{\min}), \quad (34)$$

where a is action type, s is strength, Γ is dependency expansion, and $q_{\min}$ is minimum quality. Color edits may restart late or use projection; structural edits may restart earlier.

9.2. Continuum-aware placement

A continuum version can treat each operator as a task over candidate resources $R = \{r_1, \dots, r_m\}$, such as local CPU, local accelerator, edge GPU, cloud GPU, or HPC batch node. Each operator-resource pair has an estimated tuple

$$\phi(o, r) = (T_{o,r}, E_{o,r}, C_{o,r}, M_{o,r}, P_{o,r}), \quad (35)$$

where T is time, E energy, C monetary cost, M memory risk, and P privacy risk. Placement becomes

$$(o^*, r^*) = \arg \min_{o,r} \alpha T_{o,r} + \beta D_o + \gamma Q_o + \eta E_{o,r} + \lambda C_{o,r} + \mu P_{o,r}. \quad (36)$$

The same edit may execute differently by context: a private face edit may stay local, product recolors may use projection, and high-quality SDXL structural edits may move to cloud GPUs when latency and privacy allow.

The model also supports batching: several edits can reuse one detector result, a detector backend can remain resident across images, and low-risk projections can run while diffusion jobs queue. These opportunities are hidden when each edit is a standalone prompt.

9.3. Quality-aware policy learning

The current rule-based policy is interpretable but incomplete. A learned policy could predict whether an operator will satisfy the request under a time budget from tuples

$$(\ell_i, a, s, \text{model}, \text{crop}, \text{operator}) \rightarrow (T, D, B, H), \quad (37)$$

where H is a human or model-assisted quality score. The policy could estimate a Pareto frontier: projection for speed/locality, local diffusion for realism, and naive regeneration for global coherence.

The heavy benchmark provides timing, drift, boundary, crop, and model features. Reliable semantic ground truth is still missing; human ratings or controlled VLM judgments for edit success, seam quality, identity preservation, and prompt adherence would support a data-driven scheduler.

9.4. From prototype to reusable workflow artifact

The repository packages manuscript source, scripts, reports, metrics, model-case CSVs, and selected grids. A fuller artifact would add a formal workflow specification, lockfiles or a container, and model hashes where licenses permit.

The underlying contribution is adaptive workflow execution over semantic, cached, provenance-rich intermediate state.

10. Threats to Validity

Semantic correctness. The benchmark uses detector-produced masks as operational targets. It does not yet compare against hand-labeled mask IoU or Dice scores. Therefore, the locality results measure preservation relative to the detected mask, not necessarily semantic correctness relative to human annotation.

Quality metrics. The benchmark reports time, drift, inside change, boundary change, and LPIPS. These systems and preservation metrics do not replace human evaluation, task success judgments, CLIP/BLIP alignment, or VLM-based quality assessment. An edit can preserve outside pixels and still fail semantically or visually inside the target.

Hardware generality. The 200-sample workflow ran on local Apple MPS, while the cloud revision measures NVIDIA L4 and compact T4 tiers. H100-class GPUs, edge accelerators, TensorRT deployments, batched servers, or HPC nodes may shift memory behavior, img2img throughput, and detector latency. The sensitivity projection is not a substitute for an end-to-end measured continuum scheduler.

Model set. The benchmark includes 10 compatible local checkpoints and five model families in the long L4 run. The bucket also contains Flux, Wan, Qwen, Z-image, GGUF, and ComfyUI-specific objects, but these require different runtimes and are not directly comparable to the SDXL/SD1.5 Diffusers pipelines used here.

No watt-level energy measurement or deployed multi-tier scheduler. The objective includes resource cost, but the experiment does not measure power draw or execute one workflow across coordinated resource tiers. Resource cost is a compute-work proxy from area, step count, detector invocation, and measured accelerator tier. Compute-continuum claims are limited to the representation, measured operators, GCP CUDA tiers, and sensitivity projection.

11. Conclusion

This paper presented semantic layer workflows for adaptive generative AI editing. An editing session is treated as a provenance-aware workflow over semantic layers rather than independent diffusion calls. The Image DOM stores masks, boxes, provenance, cache keys, version history, and dependency context; a policy chooses between full regeneration, local diffusion, projection, cached re-edit, and rollback.

The 200-sample local benchmark shows that semantic local rendering is faster than naive regeneration without detector overhead, but first edits are not universally faster once GroundingDINO+SAM2 is included. On an NVIDIA L4, the first-edit workflow averages 4.882 s for full-image regeneration and 1.174 s for detector-inclusive semantic editing, giving 4.28x speedup. In the long-session L4 benchmark, seven sessions across five model families run

303 variants and reduce detector-inclusive execution from 2786.7 s to 528.9 s, giving 5.37x mean speedup. Workflow-level gains are strongest: cached re-edits average 7.041x locally, deterministic projection averages 1227x for color edits, rollback averages 0.188 s, compositing preserves non-target pixels under the implemented metric, and semantic outputs keep outside-mask LPIPS near zero. These results support semantic workflow state for repeated, local, provenance-sensitive generative AI workloads.

Future work should add hand-labeled masks, CLIP/BLIP scoring, human ratings, optimized inference servers, watt-level energy measurement, and measured multi-tier scheduling. A natural next step is to persist latent denoising trajectories so the policy can restart from selected timesteps rather than only caching masks and image-space layers.

Data and code availability

The benchmark scripts, generated metrics, reports, manuscript source, and representative image artifacts are available in the public project repository:

github.com/steliosot/semantic-layer-workflows-fgcs

The repository includes benchmark summaries, model-by-case matrices, revision scripts and outputs, GCP CUDA detector-inclusive runs, L4/T4 tier results, long L4 sessions, bucket runtime and cloud LPIPS audits, selected figures, citation metadata, SHA-256 checksums, and a release archive with metrics JSON and representative sample grids. Large checkpoints and detector weights are not redistributed because they have separate licenses; the repository records filenames, Hugging Face identifiers, detector identifiers, and reproduction code.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRedit authorship contribution statement

Stelios Sotiriadis: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During manuscript preparation, the author used ChatGPT/Codex to assist with drafting, LaTeX organization, and summarization of locally produced benchmark results. The author reviewed and edited the output and takes full responsibility for the content.

Appendix A. Representative Image Generations

The public repository release archive contains the 200 benchmark sample-grid images. Each grid shows the base image, mask or overlay, naive edits, semantic local outputs, deterministic projection or cached variants where applicable, and rollback/version artifacts. They are kept in the artifact rather than embedded here to keep the manuscript within the FGCS length target.

Appendix B. Model-by-Case Matrix

The full model-by-case matrix is provided as a CSV file with the manuscript artifacts. Table B.19 reports the detector-inclusive and cached speedups by model and category in compact form.

References

- [1] E. Deelman, D. Gannon, M. Shields, I. Taylor, Workflows and e-science: An overview of workflow system features and capabilities, *Future Generation Computer Systems* 25 (5) (2009) 528–540. doi:10.1016/j.future.2008.06.012.
- [2] C. S. Liew, M. Atkinson, M. Galea, T. F. Ang, P. Martin, J. v. Hemert, Scientific workflows: moving across paradigms, *ACM Computing Surveys* 49 (4) (2017) 1–39. doi:10.1145/3012429.
- [3] R. Ferreira da Silva, R. Filgueira, I. Pietri, M. Jiang, R. Sakelariou, E. Deelman, A characterization of workflow management systems for extreme-scale applications, *Future Generation Computer Systems* 75 (2017) 228–238. doi:10.1016/j.future.2017.02.026.
- [4] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, D. Cohen-Or, Prompt-to-prompt image editing with cross-attention control, *International Conference on Learning Representations* (2023). URL https://openreview.net/forum?id=__CDixzkzeyb
- [5] L. Zhang, A. Rao, M. Agrawala, Adding conditional control to text-to-image diffusion models, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 3836–3847.
- [6] T. Brooks, A. Holynski, A. A. Efros, Instructpix2pix: Learning to follow image editing instructions, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 18392–18402.
- [7] E. Deelman, K. Vahi, M. Rynge, G. Juve, R. Mayani, S. Callaghan, P. J. Maechling, K. Wenger, W. Chen, R. Ferreira da Silva, M. Livny, et al., Pegasus, a workflow management system for science automation, *Future Generation Computer Systems* 46 (2015) 17–35. doi:10.1016/j.future.2014.10.008.
- [8] M. Satyanarayanan, The emergence of edge computing, *Computer* 50 (1) (2017) 30–39. doi:10.1109/MC.2017.9.
- [9] S. B. Davidson, J. Freire, Provenance and scientific workflows: challenges and opportunities, in: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, 2008, pp. 1345–1350. doi:10.1145/1376616.1376772.
- [10] L. Moreau, P. Missier, Prov-dm: The prov data model, W3C Recommendation (2013). URL <https://www.w3.org/TR/prov-dm/>
- [11] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, et al., The fair guiding principles for scientific data management and stewardship, *Scientific Data* 3 (2016) 160018. doi:10.1038/sdata.2016.18.
- [12] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 6840–6851.
- [13] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10684–10695.
- [14] J. Song, C. Meng, S. Ermon, Denoising diffusion implicit models, *International Conference on Learning Representations* (2021). URL <https://openreview.net/forum?id=StIgiarCHLP>
- [15] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, S. Ermon, Sdedit: Guided image synthesis and editing with stochastic differential equations, *International Conference on Learning Representations* (2022). URL https://openreview.net/forum?id=aBsCjcPu_tE
- [16] G. Couairon, J. Verbeek, H. Schwenk, M. Cord, Diffedit: Diffusion-based semantic image editing with mask guidance, *International Conference on Learning Representations* (2023). URL <https://openreview.net/forum?id=3lge0p5o-M->
- [17] R. Tang, L. Liu, A. Pandey, Z. Jiang, G. Yang, K. Kumar, P. Stenetorp, J. Lin, F. Ture, What the daam: Interpreting stable diffusion using cross attention, in: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 5644–5659. doi:10.18653/v1/2023.acl-long.310.
- [18] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, L. Zhang, Grounding dino: Marrying dino with grounded pre-training for open-set object detection, *arXiv preprint arXiv:2303.05499* (2023).
- [19] IDEA Research, Grounding dino tiny model card, Hugging Face model repository, accessed 15 July 2026 (2023). URL <https://huggingface.co/IDEA-Research/grounding-dino-tiny>
- [20] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollar, R. Girshick, Segment anything, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4015–4026.
- [21] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollar, C. Feichtenhofer, Sam 2: Segment anything in images and videos, *arXiv preprint arXiv:2408.00714* (2024).
- [22] Meta AI, Sam 2.1 hiera tiny model card, Hugging Face model repository, accessed 15 July 2026 (2025). URL <https://huggingface.co/facebook/sam2.1-hiera-tiny>
- [23] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, H. Zhao, Depth anything: Unleashing the power of large-scale unlabeled data, *arXiv preprint arXiv:2401.10891* (2024).
- [24] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, *IEEE Transactions on Parallel and Distributed Systems* 13 (3) (2002) 260–274. doi:10.1109/71.993206.
- [25] Hugging Face, Diffusers: State-of-the-art diffusion models for image, video, and audio generation, Hugging Face documentation, accessed 15 July 2026 (2026). URL <https://huggingface.co/docs/diffusers/en/index>
- [26] RunDiffusion, Juggernaut x v10 model card, Hugging Face model repository, accessed 15 July 2026 (2026). URL <https://huggingface.co/RunDiffusion/Juggernaut-X-v10>
- [27] Stable-diffusion-v1-5, Stable diffusion v1.5 model card, Hugging Face model repository, accessed 15 July 2026 (2024). URL <https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5>

Table B.19: Compact model-by-case speedup matrix. Each cell reports detector-inclusive first-edit speedup / cached re-edit speedup.

Model	Face lips	Human shirt	Object mug	Landscape sky
Deliberate v6	0.598 / 10.814	0.599 / 6.913	0.517 / 4.945	0.604 / 4.303
DreamShaper 8	1.799 / 14.314	1.793 / 5.487	1.734 / 5.793	0.608 / 5.454
DreamShaper XL 1.0	0.630 / 7.138	0.598 / 2.953	0.543 / 4.722	0.511 / 3.982
Juggernaut Reborn	0.662 / 13.084	0.646 / 5.234	0.636 / 5.279	0.628 / 3.893
Juggernaut XL v2	1.165 / 2.917	1.488 / 10.520	1.373 / 15.825	1.184 / 12.794
OpenJourney v4	1.423 / 28.999	1.669 / 11.003	1.385 / 4.827	0.506 / 4.088
RealVisXL V4.0	0.692 / 4.043	0.590 / 3.487	0.711 / 3.149	0.805 / 2.009
Realistic Vision V6.0 B1	0.634 / 11.652	0.475 / 4.511	0.472 / 5.627	0.446 / 3.924
SDXL Base 1.0	0.653 / 6.643	0.554 / 6.090	0.543 / 5.186	0.597 / 4.437
Stable Diffusion 1.5 Base	0.518 / 10.949	0.453 / 4.654	0.463 / 5.228	0.451 / 4.773

- [28] Stability AI, Stable diffusion xl base 1.0 model card, Hugging Face model repository, accessed 15 July 2026 (2023).
URL <https://huggingface.co/stabilityai/stable-diffusion-xl-base-1.0>
- [29] Lykon, Dreamshaper 8 model card, Hugging Face model repository, accessed 15 July 2026 (2023).
URL <https://huggingface.co/Lykon/dreamshaper-8>
- [30] Lykon, Dreamshaper xl 1.0 model card, Hugging Face model repository, accessed 15 July 2026 (2023).
URL <https://huggingface.co/Lykon/dreamshaper-xl-1-0>
- [31] PromptHero, Openjourney v4 model card, Hugging Face model repository, accessed 15 July 2026 (2023).
URL <https://huggingface.co/prompthero/openjourney-v4>
- [32] SG161222, Realistic vision v6.0 b1 novae model card, Hugging Face model repository, accessed 15 July 2026 (2024).
URL https://huggingface.co/SG161222/Realistic_Vision_V6.0_B1_noVAE
- [33] SG161222, Realvisxl v4.0 model card, Hugging Face model repository, accessed 15 July 2026 (2024).
URL https://huggingface.co/SG161222/RealVisXL_V4.0
- [34] XpucT, Deliberate model card, Hugging Face model repository, accessed 15 July 2026 (2024).
URL <https://huggingface.co/XpucT/Deliberate>
- [35] HyperX-Sentience, Juggernaut reborn model card, Hugging Face model repository, accessed 15 July 2026 (2024).
URL <https://huggingface.co/HyperX-Sentience/Juggernaut-Reborn>
- [36] RunDiffusion, Juggernaut xl model card, Hugging Face model repository, accessed 15 July 2026 (2026).
URL <https://huggingface.co/RunDiffusion/Juggernaut-XL>